

ML Model Evaluation

AutoGrade — Hệ thống chấm code tự động

Table of Contents

1. Introduction

2. Model Description

3. Experimental Design

3.1 Datasets

3.2 Models and Baselines for Evaluation

3.3 Performance metrics

3.4 Environment setup

4. Results5

4.1 Tổng quan các mô-đun cứng

4.2 Mô-đun testcase

4.3 Mô-đun độ phức tạp

4.4 Mô-đun gian lận và công thức điểm

4.5 Chất lượng nhận xét AI (Gemini 2.5 Flash · Groq)

5. Conclusion

1. Introduction

Tài liệu này trình bày quy trình và kết quả đánh giá (model evaluation) các mô hình/mô-đun chấm điểm của hệ thống AutoGrade — ứng dụng web chấm code tự động cho hai chế độ nộp bài: bài tự do (sinh viên tự luyện tập, nhận phản hồi) và nộp deadline (kết quả phục vụ giảng viên chấm điểm). Nội dung tài liệu tương tự phần Evaluation trong một bài báo nghiên cứu: mô tả mô hình, thiết kế thí nghiệm, bộ dữ liệu, độ đo, kết quả và kết luận.

Mục tiêu của việc đánh giá gồm ba điểm. Thứ nhất, kiểm chứng độ chính xác của các mô-đun tắt định (chạy testcase, phân tích độ phức tạp thuật toán, phát hiện gian lận, công thức tổng hợp điểm) trên bộ dữ liệu có nhãn chuẩn (ground truth) được xây dựng thủ công, bao gồm cả các ca đối kháng (adversarial) khó. Thứ hai, lượng hoá chất lượng hai loại nhận xét do LLM sinh ra — nhận xét phong cách (Gemini 2.5 Flash) và nhận xét tổng (Groq, gpt-oss-120b) — bằng phương pháp giám khảo AI độc lập (LLM-as-a-judge) chấm theo bộ tiêu chí cố định, tái lập được. Thứ ba, từ kết quả đo được, chỉ ra các điểm yếu cụ thể của pipeline, giải thích cơ chế gây lỗi và đề xuất hướng cải thiện trước khi đưa hệ thống vào sử dụng thực tế.

2. Model Description

Hệ thống AutoGrade chấm một bài nộp bằng pipeline gồm năm mô-đun, phối hợp theo luồng: kiểm tra gian lận (cổng chặn) → chạy testcase → nếu code lỗi nghiêm trọng (fatal) thì rẽ nhánh debug (chỉ chạy đánh giá phong cách); ngược lại chạy song song phân tích độ phức tạp và đánh giá phong cách, rồi tổng kết bằng mô-đun tổng hợp. Cụ thể:

1. **Mô-đun testcase (tắt định):** biên dịch/chạy bài nộp (Python, C++, Java) trong sandbox có giới hạn thời gian, so khớp output với đáp án từng testcase; đồng thời phát hiện lỗi fatal (lỗi biên dịch/cú pháp, crash toàn bộ testcase, timeout).
2. **Mô-đun độ phức tạp (tắt định, phân tích tĩnh):** phân lớp độ phức tạp thời gian của bài nộp vào 7 lớp: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$ và chậm hơn $O(n^3)$.
3. **Mô-đun gian lận (tắt định):** tính độ tương đồng (%) giữa các bài nộp trong cùng một bài tập và gắn cờ nghi gian lận khi vượt ngưỡng (mặc định 85%).
4. **Mô-đun phong cách (LLM — Gemini API, mô hình Gemini 2.5 Flash):** sinh nhận xét về phong cách lập trình (đặt tên, cấu trúc, comment...) kèm điểm phần trăm nội bộ.

5. **Mô-đun tổng hợp (LLM — Groq API, mô hình gpt-oss-120b):** tổng hợp kết quả testcase, độ phức tạp, phong cách và gian lận thành nhận xét tổng; điểm số cuối cùng được tính bằng công thức tắt định có trọng số theo từng mô-đun. Khi lời gọi LLM thất bại, hệ thống dùng câu nhận xét dự phòng (fallback).

Như vậy, đối tượng đánh giá chia làm hai nhóm có bản chất khác nhau: (a) các mô-đun cứng cho ra kết quả tắt định, có thể so trực tiếp với nhãn chuẩn; (b) các nhận xét ngôn ngữ tự nhiên do LLM sinh ra, không có đáp án duy nhất nên phải đánh giá bằng bộ tiêu chí chất lượng.

3. Experimental Design

3.1 Datasets

Bộ dữ liệu 1 — đánh giá mô-đun cứng (ground truth gán nhãn thủ công, thiết kế đối kháng):

- **Testcase:** 60 bài nộp (41 Python, 12 C++, 7 Java) phủ ba nhóm: bài đúng hoàn toàn; bài có lỗi tinh vi chỉ rút một phần testcase (lỗi off-by-one, chia nguyên, thiếu điều kiện biên, sai định dạng output...); và 8 bài lỗi fatal (lỗi biên dịch, crash toàn bộ, đệ quy vô hạn, vòng lặp vô hạn/timeout). Mỗi bài có nhãn: số testcase đạt, phần trăm đạt và cờ fatal.
- **Độ phức tạp:** 58 chương trình gán nhãn theo 7 lớp $O()$. Bộ này cố ý chứa các ca bẫy heuristic: vòng lặp số lần cố định (thực chất $O(1)$), hai con trỏ/cửa sổ trượt khẩu hao $O(n)$, vòng trong bị giấu sau lời gọi hàm phụ, thao tác thư viện có chi phí ẩn (`std::set::insert`, `list.insert(0, x)`, `heapq`), `merge sort` tự cài.
- **Gian lận:** 62 file trong 16 nhóm bài. Ca dương (sao chép) gồm: đổi tên biến, dịch định danh Việt–Anh, đảo thứ tự khối lệnh, chèn code chết, sửa nhỏ, sao chép dây chuyền và nhiều cụm sao chép trong cùng nhóm. Ca âm gồm các lời giải độc lập, trong đó có nhóm hard negative (nhiều cách cài bubble/insertion/selection sort khác nhau cho cùng một đề).
- **Công thức điểm:** 3 ca kiểm thử tắt định với điểm kỳ vọng tính tay từ trọng số của công thức.

Bộ dữ liệu 2 — đánh giá chất lượng nhận xét AI: 67 hàng dữ liệu thu từ các lượt chạy pipeline (35 hàng ở chế độ bài tự do, 32 hàng ở chế độ nộp deadline; 52 Python, 9 C++, 6 Java). Mỗi hàng cho ra một cặp nhận xét — phong cách (Gemini 2.5 Flash) và tổng (Groq) — nên bộ dữ liệu gồm 134 nhận xét, cả hai đều được chấm. Toàn bộ 134 nhận xét đều là output thật của LLM (0 hàng chạm đường dự phòng). Bộ này được chấm lặp ba lần độc lập để ước lượng sai số của phép đo, chi tiết ở mục 4.5.

3.2 Models and Baselines for Evaluation

Nhóm không so sánh trực tiếp với các mô hình state-of-the-art bên ngoài, vì các lý do sau:

- Các mô-đun cứng là thành phần nghiệp vụ đặc thù của ứng dụng (chấm theo bộ testcase riêng, phân lớp $O()$ theo 7 bậc do đề tài định nghĩa, công thức điểm nội bộ), không tồn tại benchmark công khai tương thích để so sánh một-một; thay vào đó, chuẩn đối chiếu là nhãn ground truth gán tay — đây là oracle mạnh nhất cho bài toán.
- Với phát hiện gian lận, các công cụ như MOSS/JPlag yêu cầu gửi dữ liệu ra dịch vụ ngoài và trả về thang đo khác, khó quy đổi công bằng về cùng ngưỡng; nhóm dùng phép quét ngưỡng (threshold sweep) trên chính mô-đun như một dạng so sánh giữa các điểm vận hành. Việc đối sánh với MOSS được để lại làm việc tương lai.
- Với nhận xét LLM, chưa có baseline chuẩn cho phản hồi code tiếng Việt; nhóm dùng phương pháp LLM-as-a-judge: một AI giám khảo độc lập — Claude Opus 4.8 ở chế độ suy luận mở rộng (high), khác nhà cung cấp và khác họ mô hình với cả hai mô-đun sinh nhận xét — chấm theo bộ tiêu chí cố định, lượng hoá bằng tín hiệu kiểm tra được (nhận xét có bị cắt cụt không, số câu, có trích dẫn định danh trong code không, lời khuyên có đúng chuẩn ngôn ngữ không, số liệu có mâu thuẫn dữ kiện không). Vì giám khảo LLM không tắt định, quy trình chấm được lặp ba lần độc lập trên cùng dữ liệu và cùng prompt (mỗi lần bắt đầu từ ngữ cảnh trắng, không thấy kết quả các lần trước) để báo cáo trung bình kèm độ lệch chuẩn thay vì một con số đơn lẻ. Ngoài ra hai loại nhận xét (phong cách và tổng) được so sánh chéo với nhau trong cùng điều kiện.

3.3 Performance metrics

Mô-đun testcase: tỉ lệ dự đoán đúng số testcase PASS (accuracy), sai số tuyệt đối trung bình của phần trăm testcase (MAE %), và tỉ lệ phát hiện đúng lỗi fatal.

Mô-đun độ phức tạp: accuracy phân lớp $O()$, tỉ lệ dự đoán trong phạm vi ± 1 bậc, macro-F1, precision/recall/F1 theo từng lớp và ma trận nhầm lẫn.

Mô-đun gian lận: precision, recall, F1 tại ngưỡng vận hành 85%; ROC-AUC; và đường quét F1 theo ngưỡng để xác định ngưỡng tối ưu.

Công thức điểm: so khớp chính xác điểm nhận được với điểm kỳ vọng (exact match).

Nhận xét AI: điểm trung bình thang 1–5 (5 là tốt nhất) trên 5 tiêu chí gán với chế độ bài. Bài tự do (nhận xét cho sinh viên): Dễ hiểu · Hữu ích cho việc học · Bám sát code

· Trung thực số liệu · Đúng mực & ngắn gọn. Nộp deadline (nhận xét cho giảng viên):
Giúp GV nắm nhanh chất lượng · Chỉ đúng chỗ cần kiểm tra · Khách quan · Trung thực
số liệu · Ngắn gọn, dễ đối chiếu. Mọi con số trong nhận xét phải khớp dữ kiện thật;
nhận xét dự phòng (fallback) được ghi rõ và không chấm điểm tổng.

3.4 Environment setup

- **Máy đánh giá:** laptop ASUS ExpertBook P2451FA — CPU Intel(R) Core(TM) i3-10110U @ 2,10 GHz (2 nhân / 4 luồng logic), RAM 8 GB, GPU tích hợp Intel(R) UHD Graphics (driver 30.0.101.1994, WDDM 2.7), hệ điều hành Windows 10 Pro 64-bit (Build 19044), DirectX 12.
- Lưu ý về cấu hình: các mô-đun chấm đều chạy trên CPU, không dùng GPU (hai mô-đun LLM gọi qua API bên ngoài nên không phụ thuộc phần cứng cục bộ). Giới hạn thời gian 5 giây/bài của sandbox được hiệu chỉnh trên chính cấu hình này; nếu triển khai trên máy chủ yếu hơn cần đo lại để tránh bài hợp lệ bị timeout oan, còn máy mạnh hơn có thể khiến bài chậm “lọt lưới” giới hạn.
- Phần mềm: backend Node.js + Express; sandbox chạy code bằng python3, g++ và javac với giới hạn thời gian khoảng 5 giây mỗi bài; dữ liệu lưu trên Supabase/PostgreSQL.
- Dịch vụ LLM: Gemini API — mô hình Gemini 2.5 Flash (nhận xét phong cách) và Groq API — mô hình gpt-oss-120b (nhận xét tổng).
- Bộ khung đánh giá (evaluation harness): script tự động chạy toàn bộ dữ liệu, so nhân, tính chỉ số và sinh báo cáo. Giám khảo AI (Claude Opus 4.8, chế độ suy luận mở rộng) chấm nhận xét theo bộ tiêu chí cố định nêu ở mục 3.3, lặp ba lần độc lập; kết quả báo cáo dưới dạng trung bình $\pm$ độ lệch chuẩn giữa ba lần. Các mô-đun tắt định vì vậy tái lập được chính xác khi chạy lại cùng dữ liệu, còn phần chấm nhận xét tái lập được trong biên độ sai số đã công bố ở mục 4.5.

4. Results

Kết quả được trình bày theo hai nhóm: các mô-đun cứng (4.1–4.4) đối chiếu với nhân chuẩn, và chất lượng nhận xét AI (4.5) chấm bằng giám khảo AI theo bộ tiêu chí ở mục 3.3. Với mỗi mô-đun, ngoài con số tổng hợp, phần này phân tích cơ chế đằng sau từng ca sai: sai ở khâu nào, vì sao thiết kế hiện tại dẫn đến sai như vậy, và hệ quả của cái sai đó lên điểm số của sinh viên là gì — vì cùng một tỉ lệ lỗi nhưng lỗi “sai nhẹ, sai đều” và lỗi “hiếm nhưng sai nặng” đòi hỏi cách xử lý hoàn toàn khác nhau.

4.1 Tổng quan các mô-đun cứng

Mô-đun	Chỉ số chính	Giá trị
Testcase	Chính xác số testcase PASS	98,3% (59/60)
Testcase	MAE phần trăm testcase	1,67
Testcase	Phát hiện code hỏng (fatal)	100% (8/8)
Complexity	Chính xác lớp O()	94,8% (55/58)
Complexity	Dự đoán trong ± 1 bậc	94,8%
Complexity	macro-F1	0,953
Plagiarism	F1 tại ngưỡng vận hành 85%	0,923
Plagiarism	ROC-AUC	0,987
Plagiarism	Ngưỡng F1 tối ưu	65% (F1 = 0,95)
Công thức điểm	Khớp điểm kỳ vọng	3/3 ca

Bảng 1. Tổng hợp chỉ số của các mô-đun tất định

Cần đọc các con số này trong bối cảnh bộ dữ liệu được thiết kế đối kháng: gần một nửa số mẫu là ca bẫy (lỗi tinh vi, chi phí ẩn, thủ thuật che giấu sao chép) chứ không phải bài “sạch” lấy ngẫu nhiên. Trên dữ liệu dễ, các mô-đun này gần như chắc chắn đạt 100%; vì vậy 94,8–98,3% ở đây nên hiểu là cận dưới thận trọng của chất lượng thực tế, và toàn bộ các ca sai (được phân tích chi tiết bên dưới) đều rơi vào đúng vùng khó mà bộ dữ liệu cố tình nhắm tới. Điều này cũng có nghĩa: các con số tổng hợp không che giấu lỗi hệ thống nào ở vùng dữ liệu thông thường.

4.2 Mô-đun testcase

Chỉ số “chính xác số testcase PASS” là chỉ số nghiêm ngặt: để một bài được tính đúng, toàn bộ chuỗi biên dịch → chạy → nạp input → thu output → so khớp phải hoạt động chuẩn trên từng testcase; chỉ cần một testcase bị so khớp lệch là cả bài bị tính sai. Mô-đun đạt 59/60 bài (98,3%), và MAE phần trăm testcase chỉ 1,67 điểm phần trăm. Hai con số này đọc kèm nhau cho thấy lỗi thuộc dạng “hiếm nhưng gọn”: vì mỗi testcase thường chiếm 25–33% tổng điểm testcase của một bài, nếu mô-đun sai lắt nhắt ở nhiều bài thì MAE đã phải ở mức hàng chục; MAE 1,67 nghĩa là 59 bài lệch 0% và chỉ một bài lệch lớn.

Tỉ lệ phát hiện fatal 100% (8/8) có ý nghĩa vận hành đặc biệt, vì cờ fatal quyết định rẽ nhánh pipeline: bài fatal đi vào nhánh debug (chỉ nhận xét phong cách), bài bình thường đi tiếp qua độ phức tạp và tổng hợp. Nhận nhầm fatal (false positive) sẽ tước mất của một bài chạy được toàn bộ điểm độ phức tạp và nhận xét tổng; bỏ sót fatal (false negative) lại đẩy code hỏng qua các mô-đun phía sau và sinh nhận xét vô nghĩa. Bộ dữ

liệu phủ đủ bốn con đường dẫn đến fatal — lỗi biên dịch (g++/javac), lỗi cú pháp Python (py_compile), crash toàn bộ testcase (ZeroDivisionError, đệ quy vô hạn, vector out_of_range) và vòng lặp vô hạn chạm giới hạn 5 giây — và cả bốn đường đều được bắt đúng, chứng tỏ từng cơ chế chặn của sandbox (bước biên dịch, bắt exception, đếm giờ) hoạt động độc lập và đầy đủ.

Một điểm tinh tế mà mô-đun xử lý đúng: phân biệt “chạy được nhưng sai” với “không chạy được”. Bài py-empty-output (tính xong nhưng quên print) và py-sum-wrong-format (kết quả đúng nhưng in kèm chữ “Tong =”) đều rớt 100% testcase, song không bị gắn cờ fatal — đúng như nhãn. Sự phân biệt này quan trọng về mặt sự phạm: sinh viên quên print đáng nhận phản hồi đầy đủ về độ phức tạp và phong cách (bài về logic vẫn đúng), khác hẳn sinh viên nộp code không biên dịch được.

Bài nộp	Ngôn ngữ	PASS dự đoán / nhãn	Fatal dự đoán / nhãn	Ghi chú
py-sum-correct	Python	3/3 ✓	—	Bài đúng hoàn toàn, kể cả số âm.
py-fact-range-bug	Python	2/2 ✓	—	range(1, n) thiếu n → tính (n-1)!; chỉ pass n = 0 và n = 1 — chấm đúng phần đạt.
py-sum-wrong-format	Python	0/0 ✓	—	Kết quả đúng nhưng in kèm chữ → sai định dạng, rớt mọi testcase nhưng không crash — phân biệt đúng với fatal.
cpp-timeout-loop	C++	0/0 ✓	true/true	Vòng lặp vô hạn → timeout → gắn cờ fatal đúng.
java-syntax-error	Java	0/0 ✓	true/true	Thiếu dấu chấm phẩy → lỗi biên dịch javac, bắt đúng.
cpp-avg-no-precision	C++	3/0 ✗	false/false	Ca sai duy nhất: quên setprecision/fixed nên in “2.5” thay vì “2.50”; theo nhãn phải rớt mọi testcase nhưng mô-đun tính đạt 3/3.

Bảng 2. Ví dụ tiêu biểu và ca sai duy nhất của mô-đun testcase

Ca sai duy nhất **cpp-avg-no-precision** đáng phân tích kỹ vì nó lộ ra một quyết định thiết kế chứ không phải một bug ngẫu nhiên. Bài này tính giá trị trung bình đúng nhưng quên setprecision/fixed nên in “2.5” thay vì “2.50”. Nhãn chuẩn coi đây là rớt toàn bộ (so khớp chuỗi nghiêm ngặt — đúng thông lệ chấm thi lập trình, nơi định dạng output là một phần của đề). Mô-đun lại chấm 3/3, chứng tỏ bước so khớp output đang chuẩn hoá số học: phân tích chuỗi thành số rồi so giá trị, nên “2.5” và “2.50” bị coi là một. Hệ

quả thực tế: mọi bài đúng logic nhưng sai định dạng số thập phân sẽ được điểm tối đa, trái với thang chấm của giảng viên nếu đề yêu cầu định dạng cụ thể. Hướng sửa là cho phép cấu hình theo đề: mặc định so khớp chuỗi nghiêm ngặt, chỉ bật so khớp số học (kèm dung sai epsilon) khi người ra đề chủ động chọn — nhờ vậy sự “khoan dung” trở thành lựa chọn có chủ đích thay vì hành vi ngầm định.

4.3 Mô-đun độ phức tạp

Mô-đun phân lớp đúng 55/58 chương trình (94,8%, macro-F1 = 0,953). Vì đây là phân tích tĩnh (đọc cấu trúc code, không chạy), câu hỏi thực chất của phần đánh giá này là: mô-đun “hiểu” code đến tầng nào — chỉ đếm tầng vòng lặp lồng nhau, hay nhận ra được cả chi phí không hiện ra trên bề mặt cú pháp? Kết quả trên nhóm ca bầy trả lời khá rõ: mô-đun vượt xa mức đếm vòng lặp ngây thơ. Nó nhận ra vòng lặp chạy đúng 32 hoặc 100 lần cố định là $O(1)$ (tức có kiểm tra cận lặp có phụ thuộc n hay không); tính đúng chi phí ẩn của thao tác thư viện — `std::set::insert` là $O(\log n)$ mỗi lần khiến một vòng `for` “trông tuyến tính” thành $O(n \log n)$, `list.insert(0, x)` là $O(n)$ mỗi lần khiến một vòng `for` thành $O(n^2)$; lần theo được lời gọi hàm phụ ít nhất hai tầng (vòng ngoài $O(n)$ gọi hàm phụ $O(n^2) \rightarrow O(n^3)$); và phân lớp đúng cả `merge sort` tự cài (đệ quy chia đôi + trộn tuyến tính $\rightarrow O(n \log n)$) lẫn đệ quy một nhánh chạy dọc mảng ($O(n)$).

Thật \ Dự đoán	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$>O(n^3)$
$O(1)$	7	0	0	0	0	0	0
$O(\log n)$	0	8	0	0	0	0	0
$O(n)$	0	0	8	0	1	0	0
$O(n \log n)$	0	0	0	8	0	0	0
$O(n^2)$	0	0	0	0	10	0	0
$O(n^3)$	0	0	0	0	0	7	0
$>O(n^3)$	0	0	1	0	1	0	7

Bảng 3. Ma trận nhầm lẫn phân lớp độ phức tạp dạng heatmap (hàng = nhãn thật, cột = dự đoán; ô xanh trên đường chéo = dự đoán đúng, xanh càng đậm số mẫu càng nhiều; ô đỏ ngoài đường chéo = ca dự đoán sai)

Heatmap cho thấy sai số không rải rác mà dồn về góc dưới: hai trong ba ca đỏ nằm ở hàng $>O(n^3)$ (dự đoán non tay xuống $O(n)$ và $O(n^2)$), ca còn lại ở hàng $O(n)$ (dự đoán quá tay lên $O(n^2)$). Toàn bộ vùng giữa của ma trận — từ $O(\log n)$ đến $O(n^3)$ với các bầy thư viện và hàm phụ — sạch hoàn toàn.

Lớp	Precision	Recall	F1	Số mẫu
O(1)	1,000	1,000	1,000	7
O(log n)	1,000	1,000	1,000	8
O(n)	0,889	0,889	0,889	9
O(n log n)	1,000	1,000	1,000	8
O(n ²)	0,833	1,000	0,909	10
O(n ³)	1,000	1,000	1,000	7
>O(n ³)	1,000	0,778	0,875	9

Bảng 4. Precision / Recall / F1 theo từng lớp độ phức tạp

Ba ca sai có chung một gốc rễ: độ phức tạp thật không còn nằm trong cú pháp mà nằm trong ngữ nghĩa thực thi — vùng mà phân tích tĩnh dựa trên cấu trúc, về nguyên tắc, không với tới được:

- **py-lin-sliding-window** (nhãn O(n), dự đoán O(n²), lệch +2 bậc): về cú pháp đây đúng là while lồng trong for — mọi bộ đếm tầng lồng đều sẽ nói O(n²). Muốn kết luận O(n) phải lập luận khấu hao: con trỏ trái chỉ tiến, tổng số bước tiến qua toàn bộ vòng ngoài không vượt quá n, nên tổng chi phí vòng trong là O(n) chứ không phải O(n) mỗi lần lặp. Đây là suy luận về bất biến toàn cục của biến đếm — vượt khỏi phạm vi luật cú pháp cục bộ.
- **py-exp-powerset-recursive** (nhãn >O(n³), dự đoán O(n), lệch −4 bậc): cấu trúc điều khiển chỉ là một đệ quy một nhánh sâu n tầng — nhìn cấu trúc thì đúng là “tuyến tính”. Chi phí hàm mũ nằm ở dữ liệu: danh sách kết quả nhân đôi mỗi tầng, đến tầng cuối có 2ⁿ phần tử phải sao chép. Mô-đun đo hình dạng của điều khiển nhưng không đo sự phình to của dữ liệu, nên trượt xa nhất (4 bậc).
- **py-fact-permutations** (nhãn >O(n³), dự đoán O(n²), lệch −2 bậc): một vòng for duyệt itertools.permutations — chi phí n! ẩn hoàn toàn sau một generator của thư viện chuẩn. Trường hợp này thực ra khắc phục được bằng cách mở rộng bảng chi phí thư viện (như đã làm với set::insert): permutations/combinations/product cần được gán chi phí giai thừa/tổ hợp.

Chỉ số “trong ±1 bậc” cũng bằng đúng 94,8% mang một hàm ý quan trọng: mô-đun không có ca “sai sát nút” — hoặc đúng tuyệt đối, hoặc trượt 2–4 bậc. Với bài toán chấm điểm, đây là con dao hai lưỡi. Mặt tốt: không có nhiều nhỏ lan toả (sinh viên không bị trừ vu vơ nửa bậc). Mặt xấu: khi sai thì hậu quả nặng — bài của số trượt O(n) chuẩn mực bị chấm như bài O(n²) và mất điểm tối ưu oan, trong khi bài vét cạn hàm mũ lại được điểm như bài tuyến tính. Kết hợp với ca py-fib-memo-global ở mục 4.5 (đệ quy

có memo bị xếp “chậm hơn $O(n^3)$ ”), nhóm lỗi này khoanh vùng ba mẫu hình cần bổ sung luật chuyên biệt: (i) hai con trở/cửa sổ trượt — nhận diện qua dấu hiệu con trở đơn điệu; (ii) đệ quy kèm cache/lru_cache/dict memo — hạ về số trạng thái phân biệt; (iii) generator tổ hợp của thư viện chuẩn — tra bảng chi phí. Cả ba đều là mẫu hình phổ biến trong bài tập sinh viên nên đáng đầu tư, thay vì cố xây một bộ phân tích khấu hao tổng quát (bài toán bất khả thi về lý thuyết).

4.4 Mô-đun gian lận và công thức điểm

ROC-AUC = 0,987 trả lời câu hỏi “điểm tương đồng có xếp hạng đúng không”: chọn ngẫu nhiên một cặp sao chép và một cặp độc lập, thì 98,7% số lần cặp sao chép có điểm tương đồng cao hơn. Nói cách khác, thước đo tương đồng gần như tách bạch hoàn toàn hai quần thể — chất lượng của mô-đun vì thế quy về một quyết định duy nhất: đặt ngưỡng ở đâu. Đây chính là lý do phân đánh giá dành riêng một phép quét ngưỡng thay vì chỉ báo một con số F1.

Ngưỡng %	Precision	Recall	F1
0	0,22	1,00	0,36
10	0,26	1,00	0,42
20	0,28	1,00	0,44
30	0,37	1,00	0,54
40	0,43	1,00	0,60
50	0,53	1,00	0,69
60	0,69	0,95	0,80
70	1,00	0,90	0,95
80	1,00	0,86	0,92
90	1,00	0,67	0,80
100	1,00	0,62	0,76

Bảng 5. Quét ngưỡng phát hiện gian lận: precision / recall / F1 theo ngưỡng tương đồng

Đường quét ngưỡng kể một câu chuyện có cấu trúc rõ. Dưới 60%, recall tuyệt đối nhưng precision sụp đổ — điều này phản ánh một sự thật của bài tập nhập môn: hai lời giải độc lập cho cùng một bài đơn giản (gcd, tìm kiếm nhị phân...) tự nhiên giống nhau 40–60% vì thuật toán chỉ có một dáng chuẩn; ngưỡng thấp sẽ đánh oan hàng loạt. Từ 70% trở lên, precision đạt 1,00 và giữ nguyên: không một cặp bài độc lập nào vượt 70% tương đồng, kể cả nhóm hard negative gồm bốn cách cài sắp xếp khác nhau cho cùng một đề — nhóm được thiết kế riêng để kiểm tra xem mô-đun có nhầm “giống nhau vì

cùng thuật toán” với “giống nhau vì sao chép” hay không. Kết luận vận hành: ngưỡng 85% hiện tại nằm sâu trong vùng an toàn hơn mức cần thiết — nó không mua thêm chút precision nào (đã là 1,00 từ 70%) nhưng trả giá bằng recall tụt từ 0,90 xuống 0,86. Hạ ngưỡng về vùng 65–70% tăng khả năng bắt sao chép mà không đánh oan thêm ca nào trên bộ dữ liệu này.

Ba ca lọt lưới ở ngưỡng 85% cùng phơi bày một điểm yếu cấu trúc của thước đo: độ tương đồng đang tính trên toàn văn bản, nên mẫu số có thể bị “pha loãng”. Cấp grp-fibonacci chèn code chết: phần sao chép giống hệt, nhưng khối code thừa không khớp làm tỉ lệ tụt xuống 76,47% — dưới ngưỡng. Ca grp-valid-parens/partial.py chỉ chép một đoạn của bài gốc: phần chép giống 100% nhưng chiếm thiểu số văn bản, tương đồng tổng chỉ còn 60%. Đây chính xác là hai thủ thuật né máy quét mà sinh viên hay dùng (độn code, chép một phần), nên không thể coi là ca ngoại lệ hiếm. Hai hướng khắc phục bổ trợ nhau: (i) hạ ngưỡng về 65–70% — bắt ngay được cặp code chết (76,47%) mà theo phân tích trên không tổn precision; (ii) về dài hạn, đổi cách tính từ tỉ lệ khớp toàn file sang so khớp cục bộ (tìm đoạn khớp dài nhất/căn chỉnh cục bộ) hoặc loại bỏ code không bao giờ được gọi trước khi so — khi đó cả ca chép một phần (60%) cũng hiện hình. Cũng cần nói rõ vì sao mô-đun nên tiếp tục ưu tiên precision: chi phí của hai loại lỗi không đối xứng — gán cò oan một sinh viên trung thực là cáo buộc gian lận, nặng hơn nhiều so với để lọt một ca chép; dữ liệu quét ngưỡng cho thấy may mắn là ta có thể tăng recall “miễn phí” đến tận vùng 70% mà không phải đánh đổi nguyên tắc này.

Công thức tổng hợp điểm là hàm tất định nên chỉ cần kiểm thử khớp chính xác: cả 3/3 ca cho điểm nhận được trùng khít điểm kỳ vọng tính tay, bao gồm ca có phần thập phân ($10 \rightarrow 10$; $5,3 \rightarrow 5,3$; $3 \rightarrow 3$). Ca 5,3 đáng chú ý vì nó xác nhận đồng thời ba thứ: trọng số giữa các mô-đun được ghép đúng, phép làm tròn nhất quán với đặc tả, và không có lỗi cộng dồn số thực. Khác với các mô-đun thống kê ở trên, thành phần này không cần đánh giá trên mẫu lớn — đúng/sai của nó là thuộc tính cài đặt, không phải thuộc tính xác suất.

4.5 Chất lượng nhận xét AI (Gemini 2.5 Flash · Groq)

4.5.1 Vì sao dùng giám khảo LLM, và vì sao phải chấm ba lần

Khác với bốn mô-đun ở các mục trên, hai nhận xét do LLM sinh ra không có đáp án đúng duy nhất: cùng một bài code có thể nhận nhiều nhận xét khác nhau mà vẫn tốt như nhau. Không tồn tại nhãn ground truth để so khớp, nên nhóm dùng phương pháp **LLM-as-a-judge** — một mô hình ngôn ngữ đóng vai giám khảo độc lập, chấm từng nhận xét theo bộ tiêu chí cố định ở mục 3.3. Điều này cần được nói thẳng ngay từ đầu để người

đọc hiểu đúng bản chất con số: đây là một **ước lượng do AI thực hiện**, không phải phép đo đối chiếu với chân lý.

Giám khảo được chọn là **Claude Opus 4.8, chạy ở chế độ suy luận mở rộng (high)**. Ba lý do cho lựa chọn này. Thứ nhất, tính độc lập: giám khảo khác nhà cung cấp và khác họ mô hình với cả hai mô-đun bị chấm (Gemini 2.5 Flash cho nhận xét phong cách, gpt-oss-120b qua Groq cho nhận xét tổng), nên không xảy ra tình huống mô hình tự chấm chính mình. Thứ hai, chế độ suy luận mở rộng cho phép giám khảo thực sự đọc đoạn code trước khi phán xét nhận xét — điều bắt buộc với các tiêu chí như “Bám sát code” hay “Chỉ đúng chỗ cần kiểm tra”, vốn đòi hỏi kiểm tra xem nhận xét có bỏ sót khiếm khuyết quan trọng nhất của bài hay không. Thứ ba, giám khảo được cấp cột **Dữ kiện thật** (số testcase đạt, nhãn độ phức tạp, điểm cuối) để đối chiếu từng con số xuất hiện trong nhận xét, biến tiêu chí “Trung thực số liệu” thành một phép kiểm tra máy móc thay vì một cảm nhận.

Vấn đề còn lại là: **một lượt chấm của một LLM thì chưa đủ để ai tin**. Giám khảo LLM không tất định; cùng một dữ liệu, hai lượt chạy có thể cho hai bảng điểm lệch nhau. Nếu chỉ báo cáo một con số duy nhất, người đọc không có cách nào biết con số ấy phản ánh chất lượng của hệ thống hay chỉ phản ánh tâm trạng của một lượt sinh. Vì vậy toàn bộ quy trình chấm được lặp **ba lần độc lập**: cùng bộ dữ liệu, cùng prompt giám khảo, cùng bộ tiêu chí, nhưng mỗi lần bắt đầu từ ngữ cảnh trắng và không được thấy kết quả của các lần trước. Ba lần chấm vì thế đóng vai ba giám khảo độc lập, cho phép tính trung bình, độ lệch chuẩn và biên độ dao động — tức lượng hoá được **sai số của chính phép đo**, chứ không chỉ kết quả của phép đo. Nếu ba lần cho kết quả lệch nhau lớn, con số không đáng tin và phải nói rõ như vậy; nếu ba lần hội tụ, kết luận mới có sức nặng.

Đối tượng đo gồm đúng **bốn ô** — tích của hai chế độ bài (tự do · nộp deadline) với hai loại nhận xét (phong cách · tổng): phong cách × tự do, tổng × tự do, phong cách × deadline, tổng × deadline. Bốn ô này không thể gộp vì chúng dùng hai bộ tiêu chí khác nhau và phục vụ hai đối tượng đọc khác nhau (sinh viên và giảng viên); một nhận xét ngắn gọn, khách quan có thể là 5/5 với giảng viên nhưng chỉ 3/5 với sinh viên đang cần được hướng dẫn. Mỗi lần chấm sinh ra bốn con số ô cộng một con số tổng; ba lần chấm cho ba bộ như vậy.

Quy mô mẫu: 67 hàng dữ liệu (35 bài tự do, 32 bài nộp deadline; 52 Python, 9 C++, 6 Java), mỗi hàng gồm đoạn code, cột dữ kiện thật và hai nhận xét. Như vậy mỗi lượt chấm xử lý $134 \text{ nhận xét} \times 5 \text{ tiêu chí} = 670 \text{ lượt chấm tiêu chí}$; ba lượt cộng lại là **402 nhận xét-lượt và 2.010 lượt chấm tiêu chí**. Trong cả ba lần, 0/67 hàng rơi vào đường

dự phòng (fallback) — mọi nhận xét đều là output thật của LLM nên đều được chấm; đây đồng thời là một hạn chế về độ phủ, bàn ở mục 4.5.6.

Một khác biệt thủ tục cần ghi rõ để minh bạch: ở lần chấm thứ ba, giám khảo tự đánh dấu tiêu chí “Nắm nhanh chất lượng” là **không áp dụng** cho nhận xét phong cách (lập luận: bình luận phong cách không có nhiệm vụ tóm tắt đạt/chưa đạt cho giảng viên), nên ô phong cách $\times$ deadline của lần 3 được tính trên 4 tiêu chí thay vì 5. Lần 1 và lần 2 vẫn chấm tiêu chí này và cho điểm thấp (4,06 và 4,06). Đây là một nguồn khác biệt có hệ thống giữa các lần, và nó làm ô phong cách $\times$ deadline của lần 3 cao lên; con số vẫn được giữ nguyên trong bảng dưới thay vì chỉnh sửa, và người đọc cần lưu ý điều đó khi so ba cột.

4.5.2 Kết quả ba lần chấm và độ ổn định của phép đo

Bảng 6 trình bày điểm trung bình (thang 1–5) của bốn ô đo qua ba lần chấm độc lập, kèm trung bình, độ lệch chuẩn mẫu và biên độ ($\max - \min$) giữa ba lần. Đây là bảng quan trọng nhất của mục 4.5: nó không chỉ nói hệ thống được bao nhiêu điểm, mà còn nói con số ấy chắc chắn đến mức nào.

Ô đo	Lần 1	Lần 2	Lần 3	Trung bình	Độ lệch chuẩn	Biên độ
Phong cách $\times$ tự do	4,78	4,58	4,76	4,71	0,11	0,20
Tổng $\times$ tự do	4,61	4,24	4,51	4,45	0,19	0,37
Phong cách $\times$ deadline	4,63	4,56	4,79	4,66	0,12	0,23
Tổng $\times$ deadline	4,84	4,82	4,63	4,76	0,12	0,21
Trung bình toàn cục	4,71	4,54	4,67	4,64	0,09	0,17

Bảng 6. Điểm trung bình bốn ô đo qua ba lần chấm độc lập bằng Claude Opus 4.8 (high)

Điểm trung bình toàn cục qua ba lần là **4,64/5 (92,9%)**, với độ lệch chuẩn giữa ba lần chỉ 0,09 và biên độ 0,17 điểm. Với $n = 3$ (bậc tự do 2, $t = 4,303$), khoảng tin cậy 95% của trung bình toàn cục là **4,64 $\pm$ 0,22, tức [4,42; 4,86]**. Khoảng này rộng — đó là cái giá phải trả khi chỉ có ba lần chạy, và nhóm ghi nhận thẳng thay vì giấu đi. Điều đáng nói là toàn bộ 12 phép đo ô ($4 \text{ ô} \times 3 \text{ lần}$) nằm gọn trong dải 4,24–4,84, tức chưa tới 0,6 điểm trên thang rộng 4 điểm (khoảng 15% dải đo). Nói cách khác: ba giám khảo độc lập, đọc cùng 134 nhận xét ở ba thời điểm tách biệt, đều xếp chất lượng vào cùng một vùng “khá tốt nhưng chưa xuất sắc”. Một lần chấm có thể bị nghi là ngẫu nhiên; ba lần hội tụ trong dải hẹp thì kết luận mức chất lượng khó bác bỏ.

Quan trọng hơn giá trị tuyệt đối là **thứ hạng giữa các ô có ổn định qua ba lần hay không**, vì đây mới là thứ dùng để ra quyết định kỹ thuật. Kết quả: ô **tổng $\times$ tự do** là ô

thấp nhất ở cả ba lần (4,61 · 4,24 · 4,51) — bất biến tuyệt đối. Ngược lại, ô **tổng × deadline** đứng đầu ở lần 1 và lần 2 nhưng tụt xuống thứ ba ở lần 3; hai ô phong cách đảo chỗ cho nhau giữa các lần. Suy ra một quy tắc đọc bảng: **chỉ những chênh lệch lớn hơn khoảng ±0,15 điểm mới nên diễn giải**; các chênh lệch nhỏ hơn nằm trong nhiễu của chính giám khảo. Với quy tắc này, kết luận vững duy nhất về thứ hạng là: nhận xét tổng phục vụ sinh viên (bài tự do) yếu hơn rõ rệt so với ba ô còn lại, chênh khoảng 0,2–0,3 điểm.

Về phân bố điểm thô, lần chấm thứ nhất (lần duy nhất thống kê chi tiết đến từng lượt) cho thấy 77,3% số lượt chấm đạt 5 điểm, 17,0% đạt 4 điểm, 5,4% đạt 3 điểm, 0,3% đạt 2 điểm và không có lượt nào 1 điểm. Có 28/134 nhận xét (20,9%) chứa ít nhất một tiêu chí ≤ 3 điểm — gần như toàn bộ thuộc về chiều “tính hành động” chứ không phải chiều đúng/sai của thông tin. Đây là hình dạng lỗi lành tính: nhận xét không sai, chỉ chưa đủ sắc.

4.5.3 Tiêu chí nào ổn định, tiêu chí nào không — và điều đó nói lên gì

Việc chấm ba lần cho phép tách bạch hai loại tiêu chí mà một lần chấm không phân biệt được: tiêu chí dựa trên tín hiệu kiểm tra được, và tiêu chí dựa trên phán đoán thẩm mỹ. Bảng 7 và Bảng 8 đặt hai loại này cạnh nhau.

Ô đo	Lần 1	Lần 2	Lần 3	Trung bình
Phong cách × tự do	5,00	5,00	4,97	4,99
Tổng × tự do	5,00	5,00	4,94	4,98
Phong cách × deadline	4,97	5,00	4,88	4,95
Tổng × deadline	5,00	5,00	4,91	4,97

Bảng 7. Tiêu chí “Trung thực số liệu” — tiêu chí ổn định nhất qua ba lần chấm

“Trung thực số liệu” đạt trung bình **4,97/5**, giá trị thấp nhất trong 12 phép đo là 4,88 và cao nhất là 5,00 — biên độ 0,12 điểm, gần như không dao động. Không một lần chấm nào phát hiện nhận xét bịa số testcase, bịa phần trăm hay bịa điểm. Với một hệ thống chấm bài, đây là thuộc tính quan trọng nhất: sinh viên có thể bỏ qua một lời khuyên chung chung, nhưng không thể chấp nhận một con số sai. Ba lần chấm đồng thuận ở đây không phải ngẫu nhiên — tiêu chí này có tín hiệu kiểm tra được (đối chiếu từng số với cột dữ kiện thật), nên giám khảo LLM gần như không có chỗ để dao động.

Tiêu chí (chế độ · loại nhận xét)	Lần 1	Lần 2	Lần 3	TB	SD
Hữu ích cho việc học (tự do · phong cách)	4,31	4,09	4,14	4,18	0,12
Hữu ích cho việc học (tự do · tổng)	4,09	3,60	4,00	3,90	0,26

Tiêu chí (chế độ · loại nhận xét)	Lần 1	Lần 2	Lần 3	TB	SD
Bám sát code (tự do · phong cách)	4,66	4,00	4,71	4,46	0,40
Bám sát code (tự do · tổng)	4,46	3,77	4,63	4,29	0,46
Đúng mực & ngắn gọn (tự do · tổng)	4,51	4,00	3,97	4,16	0,30
Chỉ đúng chỗ cần kiểm tra (deadline · phong cách)	4,25	3,78	4,44	4,16	0,34
Chỉ đúng chỗ cần kiểm tra (deadline · tổng)	4,28	4,19	4,75	4,41	0,30
Khách quan (deadline · tổng)	5,00	4,97	4,38	4,78	0,35
Ngắn gọn, dễ đối chiếu (deadline · tổng)	4,94	4,97	4,12	4,68	0,48

Bảng 8. Các tiêu chí thấp điểm nhất và mức dao động của chúng giữa ba lần chấm

Đọc Bảng 7 và Bảng 8 cùng nhau rút ra một kết luận có tính phương pháp luận, đáng giá hơn bản thân các con số: **độ ổn định của giám khảo LLM tỉ lệ thuận với mức độ kiểm tra được của tiêu chí**. Tiêu chí có tín hiệu khách quan (“Trung thực số liệu”: đối chiếu số) dao động 0,03–0,06 điểm. Tiêu chí đòi hỏi phán đoán về mức độ (“Bám sát code”: nhận xét có **đủ** bám sát không?) dao động 0,40–0,46 điểm, tức lớn gấp gần mười lần. “Ngắn gọn, dễ đối chiếu” của nhận xét tổng dao động tới 0,48 điểm vì ba giám khảo đặt ngưỡng “thế nào là dài dòng” khác nhau — lần 1 và lần 2 chấm gần 5,0, lần 3 khắt khe hơn nên chỉ cho 4,12.

Hệ quả thực tiễn: khi trích số vào báo cáo hay dùng số để so sánh trước/sau khi sửa prompt, chỉ nên tin các tiêu chí có tín hiệu kiểm tra được, hoặc phải chạy nhiều lượt rồi báo cáo kèm độ lệch chuẩn. Một cải thiện +0,2 điểm ở tiêu chí “Bám sát code” sau khi sửa prompt là vô nghĩa nếu chính tiêu chí ấy dao động $\pm 0,4$ giữa hai lần chấm trên cùng dữ liệu. Ngược lại, mọi thay đổi ở “Trung thực số liệu” đều đáng chú ý ngay lập tức.

Về nội dung, bốn ô hội tụ về cùng một chân dung. Điểm mạnh nằm ở các tiêu chí “bề mặt”: dễ hiểu (4,9–5,0), khách quan (4,8–5,0), ngắn gọn ở nhận xét phong cách ($\approx 4,95$), và trung thực số liệu (4,97). Điểm yếu tập trung ở đúng một chiều — **tính hành động**: “Hữu ích cho việc học” của nhận xét tổng ở bài tự do là ô thấp nhất toàn bộ phép đo (3,90), theo sau là “Bám sát code” (4,29–4,46) và “Chỉ đúng chỗ cần kiểm tra” (4,16–4,41). Ba tiêu chí này đo cùng một thứ dưới ba tên gọi: nhận xét có chỉ trúng khiếm khuyết quan trọng nhất của bài, hay chỉ nói những điều đúng nhưng ai cũng nói được.

4.5.4 Các phát hiện định tính và mức đồng thuận giữa ba lần chấm

Điểm số cho biết mức chất lượng, nhưng không cho biết cần sửa gì. Vì vậy mỗi lần chấm còn liệt kê các hàng yếu nhất kèm chẩn đoán nguyên nhân. Nhóm đối chiếu ba

danh sách này và chỉ giữ lại làm kết luận những phát hiện xuất hiện ở ít nhất hai trong ba lần; các phát hiện chỉ xuất hiện một lần được ghi lại nhưng đánh dấu là chưa đủ bằng chứng. Bảng 9 tóm tắt.

Phát hiện	L1	L2	L3	Đồng thuận
Không có ca nào bịa số liệu (testcase, %, điểm)	Có	Có	Có	3/3
Bỏ sót “except:” tràn nuốt lỗi (py-prime-swallow-exception)	Có	Có	Có	3/3
Bỏ sót dead code, import thừa (py-gcd-deadcode)	Có	Có	Có	3/3
Bỏ qua thủ thuật làm tròn “magic” (py-avg-magic-format)	Có	Có	Có	3/3
Bỏ qua rừng macro #define (cpp-sum-macro-messy, bài tự do)	Có	Có	Có	3/3
Gợi ý sai kỹ thuật: khuyên bỏ [:count]	Có	Có	Có	3/3
Gợi ý vô nghĩa: “thêm docstring cho lệnh print”	Có	Có	Có	3/3
Khuyên “đặt tên ngắn gọn hơn” khi tên đã đủ rõ	Có	Có	Có	3/3
Chép lại nhãn độ phức tạp sai của mô-đun thượng nguồn	Có	Có	Có	3/3
Nhận xét tổng (Groq) dài dòng hơn nhận xét phong cách	Có	Có	Có	3/3
Lời khuyên lặp khuôn (“đổi tên biến”, “tránh bits/stdc++.h”)	Có	Có	Có	3/3
Chẩn sai ở py-reverse-misleading-comments (chế độ tự do)	Có	Có	Không	2/3
Rò rỉ tính từ tăng bốc ở chế độ deadline	Không	Không	Có	1/3
Điểm phong cách (%) dao động trên input giống hệt	Không	Có	Không	1/3

Bảng 9. Mức đồng thuận của các phát hiện định tính qua ba lần chấm độc lập

Mười một phát hiện đạt đồng thuận tuyệt đối 3/3. Đáng chú ý nhất, cả ba giám khảo đều xếp **py-prime-swallow-exception** và **py-gcd-deadcode** vào nhóm hai hàng yếu nhất toàn mẫu, với cùng một chẩn đoán: hệ thống nhìn thấy tên biến nhưng không nhìn thấy anti-pattern. Bài đầu dùng **except:** tràn nuốt mọi ngoại lệ rồi in “NO” — một lỗi có thể khiến chương trình sai lạng lặc; cả hai nhận xét chỉ khuyên đổi tên biến **ok** thành **is_prime** và còn khen “cấu trúc rõ ràng”. Bài sau chứa import thừa, hàm không dùng và hai biến chết; nhận xét chỉ nói “thống nhất tên biến”. Đây không phải sai sót ngẫu nhiên mà là

một thiên lệch có hệ thống của prompt hiện tại: nó mời mô hình bàn về **phong cách bề mặt** trước, nên lỗi nặng bị đẩy ra khỏi giới hạn 2–3 câu.

Nhóm phát hiện thứ hai, cũng đồng thuận 3/3, là **sự khuếch đại lỗi từ mô-đun thượng nguồn**. Nhận xét tổng tin tuyệt đối vào cột dữ kiện được cấp, nên khi mô-đun độ phức tạp gán nhãn sai thì nhận xét lặp lại nguyên văn con số sai đó. Ba ví dụ được cả ba lần chấm ghi nhận: **py-freq-messy** thực chất $O(n^2)$ vì dùng **x in u** và **u.index()** bên trong vòng lặp nhưng được dán nhãn $O(n)$; **py-gcd-clean** (Euclid đệ quy, thực chất $O(\log n)$) bị dán nhãn $O(n)$; và các thao tác đảo chuỗi bằng slicing (thực chất $O(n)$) bị dán nhãn $O(1)$. Nhận xét tổng khẳng định “độ phức tạp phù hợp” trong cả ba trường hợp. Đây là một nghịch lý đo lường đáng ghi rõ: nhận xét vẫn được 5/5 ở tiêu chí “Trung thực số liệu” vì nó trung thành với dữ kiện được cấp, nhưng người đọc vẫn nhận thông tin sai. Nói cách khác, **trung thực với dữ kiện không đồng nghĩa với đúng sự thật** khi dữ kiện đầu vào đã sai. Phát hiện này liên kết trực tiếp với ba ca sai của mô-đun độ phức tạp ở mục 4.3 (cửa sổ trượt, đệ quy sinh tập lũy thừa, permutations) và với ca py-fib-memo-global — đệ quy có memo bị xếp “chậm hơn $O(n^3)$ ” — cho thấy lớp LLM hiện đóng vai bộ khuếch đại thay vì bộ chặn lỗi.

Một phát hiện chỉ đạt 2/3 và chính vì thế mà nó hữu ích. Với bài **py-reverse-misleading-comments** (comment ghi “sắp xếp/tính tổng” nhưng code thực chất đảo chuỗi), lần 1 và lần 2 đều kết luận rằng ở chế độ tự do, nhận xét tổng đã **chẵn sai** — khuyên sinh viên “chỉnh cấu trúc cho khớp mục tiêu sắp xếp tăng dần”, tức bảo sửa code theo cái comment sai, trong khi lỗi nằm ở comment; cùng bài đó ở chế độ deadline lại được chẵn đúng. Lần 3 thì ngược lại: chấm hàng này 4,90/5 và ghi nhận “bắt đúng comment sai”. Ba giám khảo đọc cùng một đoạn văn bản và không nhất trí. Kết luận rút ra không phải là “ai đúng”, mà là ranh giới độ tin cậy của phương pháp: **phán đoán ở mức một hàng đơn lẻ không đủ vững để làm bằng chứng**; chỉ những mẫu hình lặp lại trên nhiều hàng và nhiều lần chấm (như hai nhóm 3/3 ở trên) mới nên dùng để ra quyết định sửa prompt. Tương tự, hai phát hiện 1/3 — rò rỉ tính từ tăng bốc ở chế độ deadline (làm tiêu chí “Khách quan” của lần 3 tụt xuống 4,38) và dao động ± 25 điểm phần trăm của điểm phong cách trên input giống hệt — được ghi lại như cảnh báo cần kiểm chứng thêm, không phải như kết luận.

4.5.5 Đề xuất cải thiện rút ra từ ba lần chấm

Các đề xuất dưới đây chỉ dựa trên nhóm phát hiện đồng thuận 3/3, xếp theo mức tác động lên tiêu chí yếu nhất (tính hành động). Tất cả đều thực hiện được ở tầng prompt, không cần đổi kiến trúc.

- **Quét lỗi ưu tiên trước khi bàn phong cách.** Yêu cầu mô hình rà lần lượt một checklist: except trần / nuốt ngoại lệ, dead code và import thừa, comment mâu thuẫn với code, magic number và làm tròn thủ công, macro / bits/stdc++.h, nhánh điều kiện trùng lặp, mảng kích thước cố định. Nếu tìm thấy, phải nêu khiếm khuyết nặng nhất TRƯỚC mọi lời khuyên đặt tên. Đây là can thiệp duy nhất nhắm thẳng vào ô thấp nhất (tổng $\times$ tự do, 3,90 ở “Hữu ích cho việc học”).
- **Chặn gợi ý tái cấu trúc sai.** Thêm ràng buộc: chỉ đề xuất thay đổi khi bảo toàn hành vi; không khẳng định một lát cắt hay điều kiện là thừa nếu chưa chắc ràng buộc đầu vào. Cấm rõ hai lỗi khái niệm đã lặp ở cả ba lần: “docstring cho lệnh print” và “bỏ [:count] vì map đã xử lý”.
- **Cảnh giác với dữ kiện thượng nguồn.** Trong prompt của mô-đun tổng, yêu cầu: nếu code có vòng lặp lồng hoặc thao tác $O(n)$ ẩn (in, .index, .count trên list, slicing) mà nhãn độ phức tạp lại là $O(1)/O(n)$, thì thay câu “độ phức tạp phù hợp” bằng “cần kiểm tra lại độ phức tạp”. Cách này chặn lỗi ở tầng nhận xét ngay cả khi mô-đun 4.3 chưa được sửa.
- **Ép ngắn gọn và cấm câu đệm cho nhận xét tổng.** Giới hạn cứng 2–3 câu; cấm các câu động viên chung chung (“cân nhắc thêm vài kiểm thử biên”, “tiếp tục duy trì phong cách”). Ở chế độ deadline, cấm tính từ tăng bốc (“xuất sắc”, “ấn tượng”) và luôn nêu điểm cuối khi dưới 10/10.
- **Khoá một chiều thông điệp về đặt tên.** Chuẩn hoá thành “tên phải mô tả rõ nghĩa”, chỉ góp ý khi tên hiện tại thực sự gây khó đọc, và tối đa một lời khuyên đặt tên mỗi nhận xét — xoá mâu thuẫn giữa hai mô-đun (phong cách khuyên tên mô tả hơn, tổng khuyên tên ngắn hơn).
- **Đồng bộ hai chế độ.** Định nghĩa thống nhất: comment không khớp chức năng là MỘT LỖI phải nêu, không phải ý định cần chiều theo. Cấm cụm “giảng viên nên xem lại” ở bài tự do và giọng “Bạn đã...” ở bài deadline.
- **Ổn định hoá điểm phong cách.** Hạ nhiệt độ sinh, cố định rubric quy đổi phần trăm, hoặc chấm hai lượt rồi lấy trung bình, nhằm giảm dao động trên cùng một đoạn code.

4.5.6 Hạn chế của chính phép đánh giá này

Để phần đánh giá này có giá trị khoa học, các hạn chế của nó phải được nêu rõ chứ không chỉ nêu kết quả. Năm điểm sau cần đọc kèm mọi con số ở trên:

- **Giám khảo cũng là LLM, không phải chuẩn vàng.** Claude Opus 4.8 có thể chia sẻ thiên kiến với các mô hình bị chấm — chẳng hạn cả ba đều có xu hướng coi lời khuyên “đổi tên biến” là hữu ích. Ba lần chạy đo được **phương sai** của

giám khảo, nhưng không đo được **độ chệch** của giám khảo: nếu giám khảo lệch một cách nhất quán, ba lần chạy sẽ nhất quán lệch theo cùng hướng và độ lệch chuẩn nhỏ sẽ trở thành một sự yên tâm giả tạo.

- **Chưa có đối chứng người chấm.** Không có nhãn của giảng viên thật nên chưa tính được mức đồng thuận người–máy (inter-rater agreement). Bước tiếp theo bắt buộc: lấy 15–20 hàng cho 1–2 giảng viên chấm mù theo cùng bộ tiêu chí, tính hệ số đồng thuận (Cohen’s κ hoặc Krippendorff’s α) với giám khảo AI. Nếu đồng thuận thấp, toàn bộ bảng số ở trên chỉ còn giá trị tham khảo định hướng.
- **Ba lần là mức tối thiểu.** $n = 3$ đủ để tính độ lệch chuẩn nhưng cho bậc tự do chỉ bằng 2, khiến khoảng tin cậy 95% rộng tới $\pm 0,22$ điểm. Kết luận “4,64/5” nên đọc là “nằm trong khoảng 4,4–4,9”, không phải một con số chính xác đến hai chữ số thập phân.
- **Mẫu lệch và thiếu độ phủ.** 52 Python so với 9 C++ và 6 Java, nên kết luận cho C++/Java yếu về mặt thống kê. 0/67 hàng chạm đường dự phòng nên hành vi khi Gemini/Groq lỗi hoàn toàn chưa được đo. 65/67 hàng đạt 10/10, tức khả năng “bắt lỗi” của nhận xét trên bài điểm thấp gần như chưa được kiểm thử.
- **Ba lần chấm chưa hoàn toàn đồng nhất về thủ tục.** Lần 3 bỏ tiêu chí “Nắm nhanh chất lượng” cho nhận xét phong cách, làm ô phong cách $\times$ deadline của lần đó cao lên; và chỉ lần 2 kiểm tra độ ổn định của điểm phong cách phần trăm. Những khác biệt này được giữ nguyên và công bố thay vì chuẩn hoá hậu kỳ, nhưng chúng làm độ lệch chuẩn ở Bảng 6 hơi lớn hơn so với một thiết kế lặp hoàn toàn giống nhau.

4.5.7 Kết luận mục 4.5

Qua ba lần chấm độc lập bằng Claude Opus 4.8 ở chế độ suy luận mở rộng, trên 67 hàng dữ liệu và 2.010 lượt chấm tiêu chí, chất lượng nhận xét AI của AutoGrade đạt **4,64/5 (92,9%)**, **độ lệch chuẩn giữa các lần 0,09**, **khoảng tin cậy 95% là [4,42; 4,86]**. Việc lặp ba lần chính là điều biến kết quả này từ một con số khó kiểm chứng thành một phép đo có sai số công bố được: một lượt chấm đơn lẻ của một LLM khó thuyết phục ai, nhưng ba lượt độc lập hội tụ trong dải 0,17 điểm, cùng xếp một ô cố định (tổng $\times$ tự do) xuống thấp nhất và cùng chỉ ra mười một khiếm khuyết giống hệt nhau, thì kết luận không còn nhiều chỗ để tranh cãi.

Chân dung tổng thể: hệ thống **trung thực gần như tuyệt đối về số liệu (4,97/5, không một ca bịa số)**, diễn đạt dễ hiểu và phân biệt tốt hai đối tượng đọc, nhưng **yếu ở tính hành động** — nó nói những điều đúng thay vì những điều quan trọng nhất. Hai rủi ro cần xử lý trước khi triển khai: (a) bỏ sót các anti-pattern “chìm” như except trần, dead

code và làm tròn thủ công; (b) khuếch đại nguyên văn nhãn độ phức tạp sai từ mô-đun thượng nguồn, khiến một nhận xét “trung thực với dữ kiện” vẫn truyền đạt thông tin sai. Cả hai đều khắc phục được bằng chỉnh sửa prompt ở mục 4.5.5. Ở mức độ tin cậy hiện tại, nhận xét AI đủ chất lượng để dùng như một lớp phản hồi hỗ trợ cho sinh viên và một lớp tóm tắt định hướng cho giảng viên, nhưng chưa nên dùng thay thế phán đoán của giảng viên trong các quyết định chấm điểm cuối cùng.

5. Conclusion

Kết quả đánh giá cho thấy các mô hình của AutoGrade phù hợp với bài toán chấm code tự động: các mô-đun tất định đạt độ chính xác cao trên bộ dữ liệu đối kháng (testcase 98,3%, phát hiện fatal 100%, độ phức tạp 94,8% với macro-F1 0,953, gian lận ROC-AUC 0,987, công thức điểm khớp 3/3), và nhận xét LLM đạt 4,64/5 — trung bình của ba lần chấm độc lập bằng Claude Opus 4.8 (chế độ suy luận mở rộng), độ lệch chuẩn giữa các lần chỉ 0,09, khoảng tin cậy 95% là [4,42; 4,86] — với tính trung thực số liệu gần như tuyệt đối (4,97/5, không một ca bịa số). Việc lặp ba lần là điều khiến con số này đáng tin: một lượt chấm đơn lẻ của một LLM khó thuyết phục, nhưng ba lượt độc lập hội tụ trong dải 0,17 điểm và cùng chỉ ra mười một khiếm khuyết giống hệt nhau thì kết luận khó bác bỏ. Hạn chế chính gồm: nhận xét yếu ở tính hành động (ô tổng $\times$ tự do chỉ 3,90/5 ở tiêu chí “Hữu ích cho việc học”), bỏ sót các anti-pattern chìm (except trần, dead code, làm tròn thủ công) và lặp lại nguyên văn nhãn độ phức tạp sai từ mô-đun thượng nguồn; nhận xét tổng dài dòng và đôi khi đưa gợi ý sai kỹ thuật; phân tích tĩnh chưa xử lý được khấu hao, chi phí ẩn và memoization; kẻ gian lận có thể độn code chết để lách ngưỡng 85%. Hướng cải thiện: thêm bước quét anti-pattern ưu tiên trước khi bàn phong cách, buộc mô-đun tổng cảnh giác khi nhãn độ phức tạp mâu thuẫn hiển nhiên với code, ép độ dài 2–3 câu và cấm gợi ý không bảo toàn hành vi, bổ sung nhận diện memoization, hạ ngưỡng gian lận về vùng 65–70%. Song song, cần đối chứng bằng điểm giám khảo AI với một nhóm giảng viên chấm mù để đo mức đồng thuận người–máy. Với các chỉnh sửa này, hệ thống đủ điều kiện triển khai thực tế như một lớp chấm và phản hồi hỗ trợ, không thay thế phán đoán cuối cùng của giảng viên.